TOP 15 PySpark WINDOW FUNCTIONS

Interview Questions (1 to 3)

1. Find the highest salary in each department.

Scenario: Given an employee DataFrame, find the highest salary in each department.

Answer: Use row_number() over a window partitioned by department ordered by salary descending and filter row_number = 1.

```
from pyspark.sql import Window
from pyspark.sql.functions import row_number, col
w = Window.partitionBy("department").orderBy(col("salary").desc())
result = (df.withColumn("rn", row_number().over(w))
          .filter(col("rn") == 1)
          .select("department", "employee_id", "salary"))
```

Output: Highest salary employee in each department.

---

2. Get the second highest salary in each department.

Scenario: Find the employee(s) with second highest salary in each department.

Answer: Use dense_rank() over department ordered by salary descending and filter dense_rank = 2.

```
from pyspark.sql import Window
from pyspark.sql.functions import dense_rank, col
w = Window.partitionBy("department").orderBy(col("salary").desc())
result = (df.withColumn("dr", dense_rank().over(w))
          .filter(col("dr") == 2)
          .select("department", "employee_id", "salary"))
```

Output: Second highest salary employee(s) in each department.

---

3. Identify duplicate records based on business keys.

Scenario: Given a DataFrame, identify and remove duplicate records based on business key columns.

Answer: Use row_number() over partition of business key columns and keep only row_number = 1.

```
from pyspark.sql import Window
from pyspark.sql.functions import row_number, col
# business key columns: 'key1', 'key2'
w = Window.partitionBy("key1", "key2").orderBy(col("ingest_ts").desc())
dedup_df = (df.withColumn("rn", row_number().over(w))
            .filter(col("rn") == 1)
            .drop("rn"))
```

Output: DataFrame with duplicate records removed.

4. Fetch the latest record for each customer.

Scenario: Given a DataFrame, fetch the most recent record (based on updated_at) for each customer.

Answer: Use row_number() over partition by customer_id ordered by updated_at descending and filter row_number = 1.

```
from pyspark.sql import Window
from pyspark.sql.functions import row_number, col
w = Window.partitionBy("customer_id").orderBy(col("updated_at").desc())
result = (df.withColumn("rn", row_number().over(w))
          .filter(col("rn") == 1)
          .select("customer_id", "updated_at", "other_columns"))
```

Output: DataFrame with latest record for each customer.

---

5. Calculate running total of sales by date.

Scenario: Given daily sales data, calculate the running total of sales over time.

Answer: Use sum(sales) over a window ordered by date from the start to the current row.

```
from pyspark.sql import Window
from pyspark.sql.functions import sum, col
w = Window.orderBy(col("date")) \
          .rowsBetween(Window.unboundedPreceding, Window.currentRow)
result = df.withColumn("running_total", sum(col("sales")).over(w))
```

Output: DataFrame with an additional column 'running_total' showing cumulative sales till that date.

---

6. Find the difference between current row and previous row.

Scenario: Given a DataFrame of daily stock prices, calculate the difference between the current day's price and the previous day's price.

Answer: Use lag() to get the previous row's value and subtract.

```
from pyspark.sql import Window
from pyspark.sql.functions import lag, col
w = Window.orderBy(col("date"))
result = (df.withColumn("prev_price", lag(col("price")).over(w))
          .withColumn("diff", col("price") - col("prev_price"))
          .drop("prev_price"))
```

Output: DataFrame with a 'diff' column showing the difference between current price and previous price.

7. Find the difference between current row and next row.

Scenario: Given a DataFrame of daily stock prices, calculate the difference between the next day's price and the current day's price.

Answer: Use lead() to get the next row's value and subtract.

```
from pyspark.sql import Window
from pyspark.sql.functions import lead, col
w = Window.orderBy(col("date"))
result = (df.withColumn("next_price", lead(col("price")).over(w))
          .withColumn("diff", col("next_price") - col("price"))
          .drop("next_price"))
```

Output: DataFrame with a 'diff' column showing difference between next day price and current day price.

---

8. Compute month-over-month growth for revenue.

Scenario: Given monthly revenue data, calculate the month-over-month (MoM) growth percentage.

Answer: Use lag() to get previous month's revenue.

```
from pyspark.sql import Window
from pyspark.sql.functions import lag, col, round
w = Window.orderBy(col("year"), col("month"))
result = (df.withColumn("prev_revenue", lag(col("revenue")).over(w))
          .withColumn("mom_growth_pct",
           round((col("revenue") - col("prev_revenue")) / col("prev_revenue") * 100, 2))
          .drop("prev_revenue"))
```

Output: DataFrame with 'mom_growth_pct' column showing month-over-month growth percentage.

---

9. Assign sequential ranks to students based on marks.

Scenario: Given a DataFrame of students and their marks, assign ranks based on marks in descending order. Students with same marks should get the same rank.

Answer: Use rank() to assign ranks with gaps.

```
from pyspark.sql import Window
from pyspark.sql.functions import rank, col
w = Window.orderBy(col("marks").desc())
result = df.withColumn("rank", rank().over(w))
result = result.orderBy(col("rank"))
```

Output: DataFrame with a 'rank' column showing rank of each student based on marks (same marks get same rank, with gaps).

10. Find top N employees in each department.

Scenario: Given a DataFrame of employees and their salaries, find top 3 employees in each department based on salary.

Answer: Use `row_number()` over partition and filter for top N.

```
from pyspark.sql import Window
from pyspark.sql.functions import row_number, col
w = Window.partitionBy("department").orderBy(col("salary").desc())
df_top3 = (df.withColumn("rn", row_number().over(w))
           .filter(col("rn") <= 3)
           .select("department", "employee_id", "employee_name", "salary", "rn"))
```

Output: Top 3 employees (highest salary) in each department.

---

11. Calculate cumulative percentage contribution of each transaction.

Scenario: Given transaction data with amount, calculate each transaction's contribution % and cumulative contribution % within each customer.

Answer: Use running sum over total sum in the window.

```
from pyspark.sql import Window
from pyspark.sql.functions import sum as _sum, col, round
w = Window.partitionBy("customer_id").orderBy(col("txn_date").asc())
w_all = Window.partitionBy("customer_id")
df = (df.withColumn("running_amount", _sum(col("amount")).over(w))
      .withColumn("total_amount", _sum(col("amount")).over(w_all))
      .withColumn("contrib_pct", round(col("amount") / col("total_amount") * 100, 2))
      .withColumn("cumulative_contrib_pct",
                  round(col("running_amount") / col("total_amount") * 100, 2)))
```

Output: DataFrame with each transaction's % contribution and cumulative % contribution within each customer.

---

12. Detect gaps in sequence numbers.

Scenario: Given a DataFrame with sequence numbers for events per entity, find where the sequence has gaps (difference > 1).

Answer: Use `lag()` to get previous sequence and compare.

```
from pyspark.sql import Window
from pyspark.sql.functions import lag, col
w = Window.partitionBy("entity_id").orderBy(col("seq_num").asc())
df_gaps = (df.withColumn("prev_seq", lag(col("seq_num"), 1).over(w))
           .withColumn("diff", col("seq_num") - col("prev_seq"))
           .filter((col("prev_seq").isNotNull()) & (col("diff") > 1))
           .select("entity_id", "prev_seq", "seq_num", "diff"))
```

Output: Rows where sequence number has gaps greater than 1.

13. Find the first and last purchase date for each user.

Scenario: Given a DataFrame of user purchases, find the first and last purchase date for each user.

Answer: Use first() and last() over window partitioned by user_id.

```
from pyspark.sql import Window
from pyspark.sql.functions import first, last, col
w = Window.partitionBy("user_id").orderBy(col("purchase_date").asc()) \
        .rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)
result = df.withColumn("first_purchase_date", first(col("purchase_date")).over(w))\
        .withColumn("last_purchase_date", last(col("purchase_date")).over(w)) \
        .select("user_id", "first_purchase_date", "last_purchase_date").distinct()
```

Output: DataFrame with first and last purchase dates for each user.

---

14. Get the most frequent value within each group.

Scenario: Given a DataFrame of product ratings, find the most frequent rating for each product.

Answer: Use count() with group and rank the values within each product.

```
from pyspark.sql import Window
from pyspark.sql.functions import col, count, row_number
w = Window.partitionBy("product_id").orderBy(col("cnt").desc(), col("rating").asc())
temp = df.groupBy("product_id", "rating").agg(count("*").alias("cnt"))
result = temp.withColumn("rn", row_number().over(w)) \
        .filter(col("rn") == 1) \
        .select("product_id", "rating", "cnt")
```

Output: Most frequent rating (and its count) for each product.

---

15. Calculate moving average for the last 7 days.

Scenario: Given daily temperature data, calculate the 7-day moving average temperature for each day (including current day).

Answer: Use avg() over a window of the last 7 rows.

```
from pyspark.sql import Window
from pyspark.sql.functions import avg, col
w = Window.partitionBy().orderBy(col("date").asc()) \
        .rowsBetween(-6, 0)   # last 7 days including current day
result = df.withColumn("moving_avg_7d", avg(col("temperature")).over(w)) \
        .select("date", "temperature", "moving_avg_7d")
```

Output: DataFrame with 7-day moving average temperature for each date.